

Consensus Cleanup

BIP 54

Agenda

- Scope, history, and current status

Agenda

- Scope, history, and current status
- Difficulty adjustment timestamps

Agenda

- Scope, history, and current status
- Difficulty adjustment timestamps
- Worst-case block validation

Agenda

- Scope, history, and current status
- Difficulty adjustment timestamps
- Worst-case block validation
- Merkle tree ambiguity

Agenda

- Scope, history, and current status
- Difficulty adjustment timestamps
- Worst-case block validation
- Merkle tree ambiguity
- Duplicate coinbase transactions

Agenda

- Scope, history, and current status
- Difficulty adjustment timestamps
- Worst-case block validation
- Merkle tree ambiguity
- Duplicate coinbase transactions
- Impact on nodes, applications, and mining pools

Context

Four old edge cases

Timewarp

difficulty can be forced
down

Legacy sigops

pathological validation
cost

64-byte txs

Merkle leaf/node
ambiguity

Duplicate txids

future BIP30 problem

BIP 54 bundles four consensus fixes; it adds no new spending functionality.

Why bundle them?

- Each issue is independent

Why bundle them?

- Each issue is independent
- Every consensus deployment has a large fixed coordination and review cost

Why bundle them?

- Each issue is independent
- Every consensus deployment has a large fixed coordination and review cost
- Bundling amortizes that cost across several mature fixes

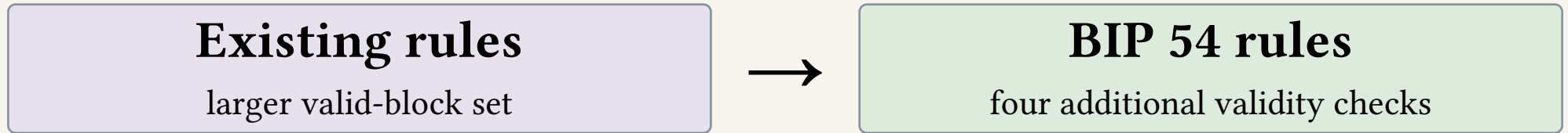
Why bundle them?

- Each issue is independent
- Every consensus deployment has a large fixed coordination and review cost
- Bundling amortizes that cost across several mature fixes
- The trade-off is atomic deployment
 - disagreement over one rule can delay all four

Why bundle them?

- Each issue is independent
- Every consensus deployment has a large fixed coordination and review cost
- Bundling amortizes that cost across several mature fixes
- The trade-off is atomic deployment
 - disagreement over one rule can delay all four
- The common theme is bounding or removing behavior nobody intends to use

It is a soft fork



Old nodes accept BIP-54-compliant blocks; upgraded nodes reject four additional cases.

History

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
- 2.
- 3.
- 4.
- 5.
- 6.

History

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot revisits the attacks and mitigations
- 3.
- 4.
- 5.
- 6.

History

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot revisits the attacks and mitigations
3. **26 March 2025** - consolidated BIP draft published
- 4.
- 5.
- 6.

History

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot revisits the attacks and mitigations
3. **26 March 2025** - consolidated BIP draft published
4. **11 April 2025** - assigned BIP 54
- 5.
- 6.

History

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot revisits the attacks and mitigations
3. **26 March 2025** - consolidated BIP draft published
4. **11 April 2025** - assigned BIP 54
5. **February 2026** - activated for testing by Bitcoin Inquisition on signet
- 6.

History

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot revisits the attacks and mitigations
3. **26 March 2025** - consolidated BIP draft published
4. **11 April 2025** - assigned BIP 54
5. **February 2026** - activated for testing by Bitcoin Inquisition on signet
6. **22 May 2026** - specification reaches version 1.0.0 and Complete status

Status today

- Authors: Antoine Poinsoy and Matt Corallo

Status today

- Authors: Antoine Poinsoy and Matt Corallo
- BIP status: **Complete**

Status today

- Authors: Antoine Poinsoot and Matt Corallo
- BIP status: **Complete**
- Complete means the specification is finished
 - it does not mean deployed or active

Status today

- Authors: Antoine Poinot and Matt Corallo
- BIP status: **Complete**
- Complete means the specification is finished
 - it does not mean deployed or active
- Active on Bitcoin Inquisition's signet deployment

Status today

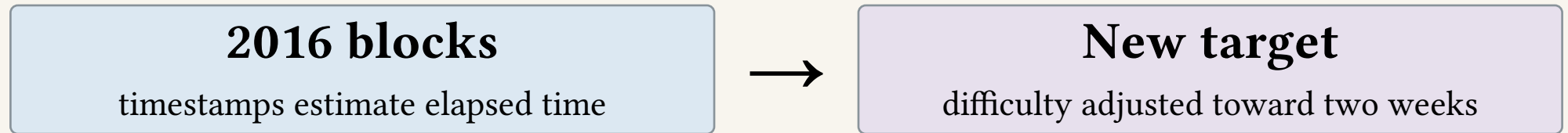
- Authors: Antoine Poinot and Matt Corallo
- BIP status: **Complete**
- Complete means the specification is finished
 - it does not mean deployed or active
- Active on Bitcoin Inquisition's signet deployment
- Bitcoin Core implementation PR is open and enables the rules only on regtest

Status today

- Authors: Antoine Poinot and Matt Corallo
- BIP status: **Complete**
- Complete means the specification is finished
 - it does not mean deployed or active
- Active on Bitcoin Inquisition's signet deployment
- Bitcoin Core implementation PR is open and enables the rules only on regtest
- No mainnet activation mechanism, parameters, or date have been selected

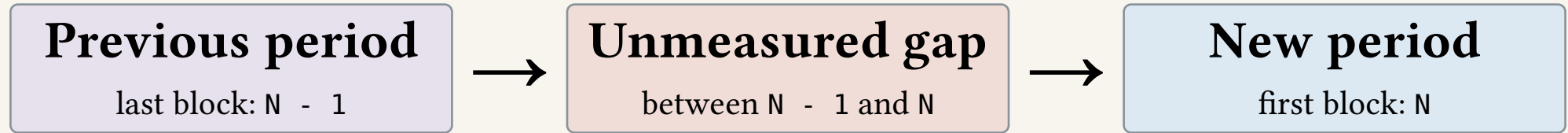
Difficulty adjustment

Retarget periods



The calculation uses the first and last timestamps inside the period.

The boundary is omitted



The first timestamp of a new period can move backward relative to the previous block.

Timewarp

- A majority-hashrate attacker manipulates timestamps across period boundaries

Timewarp

- A majority-hashrate attacker manipulates timestamps across period boundaries
- The new period begins with a timestamp earlier than the previous period ended

Timewarp

- A majority-hashrate attacker manipulates timestamps across period boundaries
- The new period begins with a timestamp earlier than the previous period ended
- Its measured duration is therefore longer than its real duration

Timewarp

- A majority-hashrate attacker manipulates timestamps across period boundaries
- The new period begins with a timestamp earlier than the previous period ended
- Its measured duration is therefore longer than its real duration
- The next retarget lowers difficulty too far

Timewarp

- A majority-hashrate attacker manipulates timestamps across period boundaries
- The new period begins with a timestamp earlier than the previous period ended
- Its measured duration is therefore longer than its real duration
- The next retarget lowers difficulty too far
- Repeating this can drive difficulty to the minimum in roughly 38 days

Timewarp

- A majority-hashrate attacker manipulates timestamps across period boundaries
- The new period begins with a timestamp earlier than the previous period ended
- Its measured duration is therefore longer than its real duration
- The next retarget lowers difficulty too far
- Repeating this can drive difficulty to the minimum in roughly 38 days
- Consequence: arbitrarily fast blocks, accelerated subsidy issuance, and broken timelock assumptions

Boundary rules

First block of period

$$T_N \geq T_{N-1} - 7200$$

Last block of period

$$T_N \geq T_{N-2015}$$

Two fixes in one

- The first-period rule limits the backward jump to two hours

Two fixes in one

- The first-period rule limits the backward jump to two hours
- That closes the repeatable timewarp mechanism

Two fixes in one

- The first-period rule limits the backward jump to two hours
- That closes the repeatable timewarp mechanism
- The last-period rule prevents a negative measured period

Two fixes in one

- The first-period rule limits the backward jump to two hours
- That closes the repeatable timewarp mechanism
- The last-period rule prevents a negative measured period
- The latter addresses the related Murch-Zawy edge case

Two fixes in one

- The first-period rule limits the backward jump to two hours
- That closes the repeatable timewarp mechanism
- The last-period rule prevents a negative measured period
- The latter addresses the related Murch-Zawy edge case
- Only the first and last blocks of each 2016-block period receive new constraints

Two fixes in one

- The first-period rule limits the backward jump to two hours
- That closes the repeatable timewarp mechanism
- The last-period rule prevents a negative measured period
- The latter addresses the related Murch-Zawy edge case
- Only the first and last blocks of each 2016-block period receive new constraints
- The two-hour grace period tolerates imperfect mining timestamp software

Validation time

Sigops

- A sigop is an occurrence of a signature-checking opcode in Bitcoin Script

Sigops

- A sigop is an occurrence of a signature-checking opcode in Bitcoin Script
- CHECKSIG usually represents one signature check

Sigops

- A sigop is an occurrence of a signature-checking opcode in Bitcoin Script
- CHECKSIG usually represents one signature check
- CHECKMULTISIG can represent several

Sigops

- A sigop is an occurrence of a signature-checking opcode in Bitcoin Script
- CHECKSIG usually represents one signature check
- CHECKMULTISIG can represent several
- Bitcoin already limits sigops at the block level

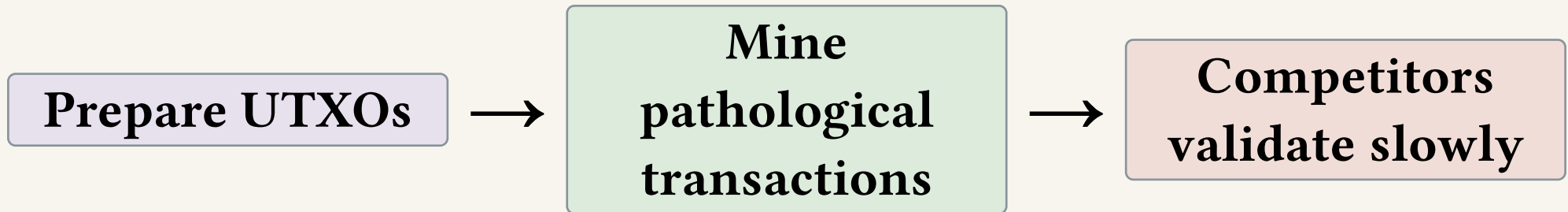
Sigops

- A sigop is an occurrence of a signature-checking opcode in Bitcoin Script
- CHECKSIG usually represents one signature check
- CHECKMULTISIG can represent several
- Bitcoin already limits sigops at the block level
- Legacy Script permits pathological transactions that concentrate expensive work

Sigops

- A sigop is an occurrence of a signature-checking opcode in Bitcoin Script
- CHECKSIG usually represents one signature check
- CHECKMULTISIG can represent several
- Bitcoin already limits sigops at the block level
- Legacy Script permits pathological transactions that concentrate expensive work
- Their validation cost is far outside ordinary wallet behavior

Slow blocks



A miner can begin hashing the next block while competitors are still validating the previous one.

Measured worst case

- Demonstrated constructions take minutes to validate on modern hardware

Measured worst case

- Demonstrated constructions take minutes to validate on modern hardware
- A 2026 measurement was about 10 minutes on a Core i5-12500H

Measured worst case

- Demonstrated constructions take minutes to validate on modern hardware
- A 2026 measurement was about 10 minutes on a Core i5-12500H
- A comparable demonstration took about 29 minutes on an older Xeon

Measured worst case

- Demonstrated constructions take minutes to validate on modern hardware
- A 2026 measurement was about 10 minutes on a Core i5-12500H
- A comparable demonstration took about 29 minutes on an older Xeon
- The absolute worst case matters for denial of service

Measured worst case

- Demonstrated constructions take minutes to validate on modern hardware
- A 2026 measurement was about 10 minutes on a Core i5-12500H
- A comparable demonstration took about 29 minutes on an older Xeon
- The absolute worst case matters for denial of service
- The cost ratio also matters
 - a miner may profit from imposing more validation work than it spent preparing

Measured worst case

- Demonstrated constructions take minutes to validate on modern hardware
- A 2026 measurement was about 10 minutes on a Core i5-12500H
- A comparable demonstration took about 29 minutes on an older Xeon
- The absolute worst case matters for denial of service
- The cost ratio also matters
 - a miner may profit from imposing more validation work than it spent preparing
- This creates a mining centralization incentive

Per-transaction limit

- Every non-coinbase transaction may contain at most **2500 legacy sigops**

Per-transaction limit

- Every non-coinbase transaction may contain at most **2500 legacy sigops**
- Count across each input's:
 - `scriptSig`
 - spent output's `scriptPubKey`
 - P2SH `redeemScript`

Per-transaction limit

- Every non-coinbase transaction may contain at most **2500 legacy sigops**
- Count across each input's:
 - `scriptSig`
 - spent output's `scriptPubKey`
 - P2SH `redeemScript`
- Count opcodes whether or not execution reaches them

Per-transaction limit

- Every non-coinbase transaction may contain at most **2500 legacy sigops**
- Count across each input's:
 - `scriptSig`
 - spent output's `scriptPubKey`
 - P2SH `redeemScript`
- Count opcodes whether or not execution reaches them
- This is the established BIP16 counting method

Per-transaction limit

- Every non-coinbase transaction may contain at most **2500 legacy sigops**
- Count across each input's:
 - `scriptSig`
 - spent output's `scriptPubKey`
 - P2SH `redeemScript`
- Count opcodes whether or not execution reaches them
- This is the established BIP16 counting method
- Witness-script sigops use their existing accounting and are not part of this new limit

Counting rules

Opcode form	BIP 54 sigops
CHECKSIG / CHECKSIGVERIFY	1
OP_1...OP_16 then CHECKMULTISIG	1...16
Other CHECKMULTISIG forms	20
Transaction total	0...2500 valid; >2500 invalid

Why 2500?

- It is the tightest studied limit that preserves non-pathological standard transactions

Why 2500?

- It is the tightest studied limit that preserves non-pathological standard transactions
- Ordinary multisig transactions hit existing size and policy limits first

Why 2500?

- It is the tightest studied limit that preserves non-pathological standard transactions
- Ordinary multisig transactions hit existing size and policy limits first
- Exceeding 2500 without exceeding transaction-size policy requires contrived scripts

Why 2500?

- It is the tightest studied limit that preserves non-pathological standard transactions
- Ordinary multisig transactions hit existing size and policy limits first
- Exceeding 2500 without exceeding transaction-size policy requires contrived scripts
- The rule targets repeated signature work rather than disabling Script features

Why 2500?

- It is the tightest studied limit that preserves non-pathological standard transactions
- Ordinary multisig transactions hit existing size and policy limits first
- Exceeding 2500 without exceeding transaction-size policy requires contrived scripts
- The rule targets repeated signature work rather than disabling Script features
- Estimated result: roughly 40 times lower worst-case block validation time

Why 2500?

- It is the tightest studied limit that preserves non-pathological standard transactions
- Ordinary multisig transactions hit existing size and policy limits first
- Exceeding 2500 without exceeding transaction-size policy requires contrived scripts
- The rule targets repeated signature work rather than disabling Script features
- Estimated result: roughly 40 times lower worst-case block validation time
- It also raises preparation cost enough to remove the plausible mining advantage

Merkle trees

Transactions and inner nodes

Transaction leaf

$\text{txid} = \text{SHA256d}(\text{serialized_tx})$

Inner node

$\text{SHA256d}(\text{left_hash} \parallel \text{right_hash})$

An inner-node preimage is always $32 + 32 = 64$ bytes.

The 64-byte ambiguity

Exactly 64 witness-stripped bytes
can be parsed as a transaction serialization
or as two child hashes of a Merkle node

Bitcoin's Merkle construction does not domain-separate leaves from inner nodes.

Consequences

- A Merkle proof can exploit the missing distinction between leaf and inner node

Consequences

- A Merkle proof can exploit the missing distinction between leaf and inner node
- An SPV verifier can be shown an apparent transaction inclusion that is not real

Consequences

- A Merkle proof can exploit the missing distinction between leaf and inner node
- An SPV verifier can be shown an apparent transaction inclusion that is not real
- Other systems consuming Bitcoin inclusion proofs inherit the same footgun

Consequences

- A Merkle proof can exploit the missing distinction between leaf and inner node
- An SPV verifier can be shown an apparent transaction inclusion that is not real
- Other systems consuming Bitcoin inclusion proofs inherit the same footgun
- Related block-validation cache bugs have existed in Bitcoin Core

Consequences

- A Merkle proof can exploit the missing distinction between leaf and inner node
- An SPV verifier can be shown an apparent transaction inclusion that is not real
- Other systems consuming Bitcoin inclusion proofs inherit the same footgun
- Related block-validation cache bugs have existed in Bitcoin Core
- Workarounds exist
 - but every proof verifier must know and implement one correctly

The rule

A transaction is invalid when its witness-stripped serialization is **exactly 64 bytes.**

- 63 bytes remains valid

The rule

A transaction is invalid when its witness-stripped serialization is **exactly 64 bytes.**

- 63 bytes remains valid
- 65 bytes remains valid

The rule

A transaction is invalid when its witness-stripped serialization is **exactly 64 bytes.**

- 63 bytes remains valid
- 65 bytes remains valid
- Existing Bitcoin Core versions neither relay nor template 64-byte transactions

The rule

A transaction is invalid when its witness-stripped serialization is **exactly 64 bytes.**

- 63 bytes remains valid
- 65 bytes remains valid
- Existing Bitcoin Core versions neither relay nor template 64-byte transactions
- They have not appeared on-chain since 2016

The rule

A transaction is invalid when its witness-stripped serialization is **exactly 64 bytes**.

- 63 bytes remains valid
- 65 bytes remains valid
- Existing Bitcoin Core versions neither relay nor template 64-byte transactions
- They have not appeared on-chain since 2016
- The proposal turns an existing policy restriction into consensus

The trade-off

- Benefit: proof verifiers no longer need special protection for this ambiguity

The trade-off

- Benefit: proof verifiers no longer need special protection for this ambiguity
- Cost: a surprising discontinuity in the valid transaction-size range

The trade-off

- Benefit: proof verifiers no longer need special protection for this ambiguity
- Cost: a surprising discontinuity in the valid transaction-size range
- Such a transaction can only create an anyone-can-spend or provably unspendable output today

The trade-off

- Benefit: proof verifiers no longer need special protection for this ambiguity
- Cost: a surprising discontinuity in the valid transaction-size range
- Such a transaction can only create an anyone-can-spend or provably unspendable output today
- Alternative proposals modify Merkle-proof or inner-node validation instead

The trade-off

- Benefit: proof verifiers no longer need special protection for this ambiguity
- Cost: a surprising discontinuity in the valid transaction-size range
- Such a transaction can only create an anyone-can-spend or provably unspendable output today
- Alternative proposals modify Merkle-proof or inner-node validation instead
- BIP 54 chooses the smallest full-node rule and removes the problematic leaf case

Coinbase uniqueness

Coinbase transactions

- Every block begins with one coinbase transaction

Coinbase transactions

- Every block begins with one coinbase transaction
- It creates the block subsidy and collects transaction fees

Coinbase transactions

- Every block begins with one coinbase transaction
- It creates the block subsidy and collects transaction fees
- Its input does not spend an earlier output

Coinbase transactions

- Every block begins with one coinbase transaction
- It creates the block subsidy and collects transaction fees
- Its input does not spend an earlier output
- Pools vary its data to produce new Merkle roots and hashing work

Coinbase transactions

- Every block begins with one coinbase transaction
- It creates the block subsidy and collects transaction fees
- Its input does not spend an earlier output
- Pools vary its data to produce new Merkle roots and hashing work
- Like every transaction, it has a txid

Coinbase transactions

- Every block begins with one coinbase transaction
- It creates the block subsidy and collects transaction fees
- Its input does not spend an earlier output
- Pools vary its data to produce new Merkle roots and hashing work
- Like every transaction, it has a txid
- Historically, some coinbase transactions were duplicated

BIP30 and BIP34

- **BIP30:** do not overwrite still-unspent outputs from an older transaction with the same txid

BIP30 and BIP34

- **BIP30**: do not overwrite still-unspent outputs from an older transaction with the same txid
- Two historical blocks are explicit exceptions

BIP30 and BIP34

- **BIP30:** do not overwrite still-unspent outputs from an older transaction with the same txid
- Two historical blocks are explicit exceptions
- **BIP34:** place the block height in the coinbase scriptSig

BIP30 and BIP34

- **BIP30:** do not overwrite still-unspent outputs from an older transaction with the same txid
- Two historical blocks are explicit exceptions
- **BIP34:** place the block height in the coinbase scriptSig
- This made modern coinbase transactions practically unique

BIP30 and BIP34

- **BIP30**: do not overwrite still-unspent outputs from an older transaction with the same txid
- Two historical blocks are explicit exceptions
- **BIP34**: place the block height in the coinbase scriptSig
- This made modern coinbase transactions practically unique
- Bitcoin Core can therefore skip most explicit BIP30 checks

BIP30 and BIP34

- **BIP30:** do not overwrite still-unspent outputs from an older transaction with the same txid
- Two historical blocks are explicit exceptions
- **BIP34:** place the block height in the coinbase scriptSig
- This made modern coinbase transactions practically unique
- Bitcoin Core can therefore skip most explicit BIP30 checks
- But BIP34 does not prove uniqueness forever

The future collision window

- Early pre-BIP34 coinbases sometimes begin with bytes that can later encode a block height

The future collision window

- Early pre-BIP34 coinbases sometimes begin with bytes that can later encode a block height
- At specific future heights, a new coinbase could reproduce an old txid

The future collision window

- Early pre-BIP34 coinbases sometimes begin with bytes that can later encode a block height
- At specific future heights, a new coinbase could reproduce an old txid
- The earliest such height is **1,983,702**

The future collision window

- Early pre-BIP34 coinbases sometimes begin with bytes that can later encode a block height
- At specific future heights, a new coinbase could reproduce an old txid
- The earliest such height is **1,983,702**
- Nodes would need to resume BIP30 validation before then

The future collision window

- Early pre-BIP34 coinbases sometimes begin with bytes that can later encode a block height
- At specific future heights, a new coinbase could reproduce an old txid
- The earliest such height is **1,983,702**
- Nodes would need to resume BIP30 validation before then
- That adds validation state and complicates alternative node designs such as Utreexo

The future collision window

- Early pre-BIP34 coinbases sometimes begin with bytes that can later encode a block height
- At specific future heights, a new coinbase could reproduce an old txid
- The earliest such height is **1,983,702**
- Nodes would need to resume BIP30 validation before then
- That adds validation state and complicates alternative node designs such as Utreexo
- BIP 54 makes future coinbase uniqueness explicit instead

Coinbase rule

nLockTime

$N - 1$

input nSequence

must not equal 0xffffffff

Together these make the coinbase valid at height N , but not at any earlier height.

Why height minus one?

- Bitcoin locktime records the last height at which a transaction is still locked

Why height minus one?

- Bitcoin locktime records the last height at which a transaction is still locked
- A transaction with `nLockTime = N - 1` is final in block N

Why height minus one?

- Bitcoin locktime records the last height at which a transaction is still locked
- A transaction with `nLockTime = N - 1` is final in block N
- It could not have been valid in any earlier block

Why height minus one?

- Bitcoin locktime records the last height at which a transaction is still locked
- A transaction with `nLockTime = N - 1` is final in block N
- It could not have been valid in any earlier block
- A non-final `nSequence` makes the locktime effective

Why height minus one?

- Bitcoin locktime records the last height at which a transaction is still locked
- A transaction with `nLockTime = N - 1` is final in block N
- It could not have been valid in any earlier block
- A non-final `nSequence` makes the locktime effective
- The height becomes directly readable from a fixed transaction field

Why height minus one?

- Bitcoin locktime records the last height at which a transaction is still locked
- A transaction with `nLockTime = N - 1` is final in block N
- It could not have been valid in any earlier block
- A non-final `nSequence` makes the locktime effective
- The height becomes directly readable from a fixed transaction field
- Therefore a post-activation coinbase cannot duplicate a historical one

Operational impact

Validity changes

Area	New invalid case	Normal activity
Difficulty	Malformed period-boundary timestamp	Unchanged
Script	Transaction above 2500 legacy sigops	Unaffected
Serialization	Exactly 64 stripped bytes	Unaffected
Coinbase	Wrong locktime or final sequence	Pool update required

Full nodes

- Upgraded nodes enforce all four rules after activation

Full nodes

- Upgraded nodes enforce all four rules after activation
- Unupgraded nodes still accept compliant blocks

Full nodes

- Upgraded nodes enforce all four rules after activation
- Unupgraded nodes still accept compliant blocks
- They do not independently reject a violating block

Full nodes

- Upgraded nodes enforce all four rules after activation
- Unupgraded nodes still accept compliant blocks
- They do not independently reject a violating block
- The sigops rule bounds adversarial validation work

Full nodes

- Upgraded nodes enforce all four rules after activation
- Unupgraded nodes still accept compliant blocks
- They do not independently reject a violating block
- The sigops rule bounds adversarial validation work
- The Merkle rule simplifies assumptions for proof-verifying applications

Full nodes

- Upgraded nodes enforce all four rules after activation
- Unupgraded nodes still accept compliant blocks
- They do not independently reject a violating block
- The sigops rule bounds adversarial validation work
- The Merkle rule simplifies assumptions for proof-verifying applications
- The coinbase rule lets implementations permanently avoid future BIP30 checks

Wallets and applications

- Ordinary wallets do not need a migration

Wallets and applications

- Ordinary wallets do not need a migration
- Non-pathological standard transactions remain valid

Wallets and applications

- Ordinary wallets do not need a migration
- Non-pathological standard transactions remain valid
- Script authors should avoid pathological legacy constructions above 2500 sigops

Wallets and applications

- Ordinary wallets do not need a migration
- Non-pathological standard transactions remain valid
- Script authors should avoid pathological legacy constructions above 2500 sigops
- Transaction generators must not produce exactly 64 witness-stripped bytes

Wallets and applications

- Ordinary wallets do not need a migration
- Non-pathological standard transactions remain valid
- Script authors should avoid pathological legacy constructions above 2500 sigops
- Transaction generators must not produce exactly 64 witness-stripped bytes
- SPV, bridge, and sidechain software benefits from removing the Merkle ambiguity

Wallets and applications

- Ordinary wallets do not need a migration
- Non-pathological standard transactions remain valid
- Script authors should avoid pathological legacy constructions above 2500 sigops
- Transaction generators must not produce exactly 64 witness-stripped bytes
- SPV, bridge, and sidechain software benefits from removing the Merkle ambiguity
- No address format, key, signature algorithm, or transaction relay protocol changes

Mining pools

- Use curtime or mintime from getblocktemplate

Mining pools

- Use `curtime` or `mintime` from `getblocktemplate`
- Exclude transactions violating the new consensus rules

Mining pools

- Use `curtime` or `mintime` from `getblocktemplate`
- Exclude transactions violating the new consensus rules
- Construct coinbase with:
 - `nLockTime = block height - 1`
 - `input nSequence != 0xffffffff`

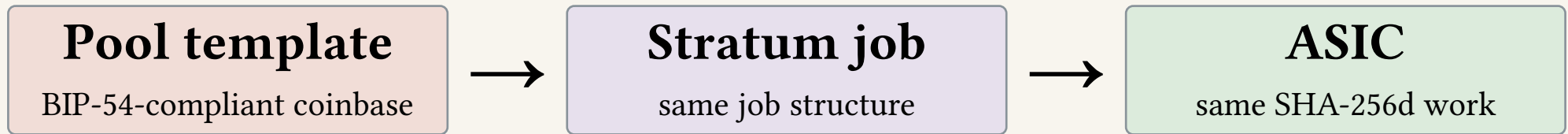
Mining pools

- Use `curtime` or `mintime` from `getblocktemplate`
- Exclude transactions violating the new consensus rules
- Construct coinbase with:
 - `nLockTime = block height - 1`
 - `input nSequence != 0xffffffff`
- The extranonce remains available in the coinbase `scriptSig`

Mining pools

- Use `curtime` or `mintime` from `getblocktemplate`
- Exclude transactions violating the new consensus rules
- Construct coinbase with:
 - `nLockTime = block height - 1`
 - `input nSequence != 0xffffffff`
- The extranonce remains available in the coinbase `scriptSig`
- Pool software, not the ASIC, normally constructs these fields

Stratum and hashing



No Stratum protocol or proof-of-work algorithm change is required.

Forward compatibility already present

- Bitcoin Core 29.0+ templates comply with the timestamp restriction

Forward compatibility already present

- Bitcoin Core 29.0+ templates comply with the timestamp restriction
- Bitcoin Core 30.0+ policy excludes transactions above the new sigops limit

Forward compatibility already present

- Bitcoin Core 29.0+ templates comply with the timestamp restriction
- Bitcoin Core 30.0+ policy excludes transactions above the new sigops limit
- Bitcoin Core 0.16.1+ excludes 64-byte stripped transactions from relay and templates

Forward compatibility already present

- Bitcoin Core 29.0+ templates comply with the timestamp restriction
- Bitcoin Core 30.0+ policy excludes transactions above the new sigops limit
- Bitcoin Core 0.16.1+ excludes 64-byte stripped transactions from relay and templates
- The remaining practical integration point is pool coinbase construction

Forward compatibility already present

- Bitcoin Core 29.0+ templates comply with the timestamp restriction
- Bitcoin Core 30.0+ policy excludes transactions above the new sigops limit
- Bitcoin Core 0.16.1+ excludes 64-byte stripped transactions from relay and templates
- The remaining practical integration point is pool coinbase construction
- These policies reduce activation risk but do not activate BIP 54

Deployment remains separate

- BIP 54 specifies post-activation validity rules

Deployment remains separate

- BIP 54 specifies post-activation validity rules
- It deliberately does not select how or when mainnet activates them

Deployment remains separate

- BIP 54 specifies post-activation validity rules
- It deliberately does not select how or when mainnet activates them
- The Bitcoin Core implementation under review contains no mainnet activation

Deployment remains separate

- BIP 54 specifies post-activation validity rules
- It deliberately does not select how or when mainnet activates them
- The Bitcoin Core implementation under review contains no mainnet activation
- Activation requires a separate coordination decision

Deployment remains separate

- BIP 54 specifies post-activation validity rules
- It deliberately does not select how or when mainnet activates them
- The Bitcoin Core implementation under review contains no mainnet activation
- Activation requires a separate coordination decision
- Until then, mainnet consensus is unchanged

Takeaways

- BIP 54 is a four-part consensus hardening soft fork

Takeaways

- BIP 54 is a four-part consensus hardening soft fork
- It limits timewarp, pathological legacy sigops, and Merkle ambiguity

Takeaways

- BIP 54 is a four-part consensus hardening soft fork
- It limits timewarp, pathological legacy sigops, and Merkle ambiguity
- It also makes future coinbase uniqueness explicit

Takeaways

- BIP 54 is a four-part consensus hardening soft fork
- It limits timewarp, pathological legacy sigops, and Merkle ambiguity
- It also makes future coinbase uniqueness explicit
- Ordinary transactions, wallets, Stratum jobs, and SHA-256d mining remain unchanged

Takeaways

- BIP 54 is a four-part consensus hardening soft fork
- It limits timewarp, pathological legacy sigops, and Merkle ambiguity
- It also makes future coinbase uniqueness explicit
- Ordinary transactions, wallets, Stratum jobs, and SHA-256d mining remain unchanged
- Pools must update coinbase locktime and sequence before activation

Takeaways

- BIP 54 is a four-part consensus hardening soft fork
- It limits timewarp, pathological legacy sigops, and Merkle ambiguity
- It also makes future coinbase uniqueness explicit
- Ordinary transactions, wallets, Stratum jobs, and SHA-256d mining remain unchanged
- Pools must update coinbase locktime and sequence before activation
- The specification is complete; mainnet deployment is not decided

Operational impact

Q&A